Refactoring: Fondamenti

Il refactoring è **chirurgia sul codice vivo**. Non aggiungi organi nuovi — risani quelli esistenti senza fermare il paziente.

La definizione precisa di Martin Fowler: *modificare la struttura interna del codice senza cambiarne il comportamento osservabile dall'esterno.*

Perché farlo?

Il codice accumula **debito tecnico** — ogni scorciatoia presa oggi è un interesse da pagare domani. Il refactoring estingue quel debito progressivamente.

Tre nemici principali che il refactoring combatte:

Leggibilità degradata → nomi ambigui, metodi troppo lunghi, logica oscura
Accoppiamento eccessivo → classi che sanno troppo delle altre **Responsabilità multiple** → una classe che fa tre cose invece di una (viola SRP)

Il processo corretto

1. Assicurati che il codice funzioni (baseline)
2. Identifica un singolo problema
3. Modifica SOLO quello
4. Verifica che funzioni ancora
5. Ripeti

Il punto critico è **step 2** → **4** → **5**. Mai fare più cambiamenti in parallelo senza verificare. In Unity questo significa: Play → testa → modifica → Play → testa.

Refactoring di Progetto Survival Runner

Cosa sappiamo dalla struttura

Il progetto ha **4 domini** impliciti:

Gameplay → Coin, Door, Camera, Enemy
Player → Controller, Health
UI → HealthBar, MenuNavigation
Management → GameEvents, GameManager

File `GameEvents.cs`

Problema originale GameEvents veniva trovato dagli altri script tramite `FindObjectOfType<GameEvents>()`. Questo metodo itera su tutti i GameObject della scena ogni volta che viene chiamato — costoso a runtime e fragile.

Soluzione: Singleton Pattern GameEvents è l'unico candidato naturale al Singleton perché soddisfa tutti e tre i criteri: deve esistere in una sola istanza, per tutta la durata del gioco, accessibile globalmente. GameManager invece no — è legato alla scena di gioco e viene distrutto quando si torna al menu.

Con il Singleton, `FindObjectOfType` scompare da tutti gli script. L'accesso diventa `GameEvents.Instance` — sempre disponibile, O(1).

Problema introdotto dal DontDestroyOnLoad GameEvents sopravvive tra le scene, ma i listener iscritti dal vecchio GameManager resterebbero appesi a oggetti distrutti — causando memory leak e potenzialmente comportamento doppio (ogni evento processato due volte).

Soluzione: reset automatico al cambio scena GameEvents si iscrive a `SceneManager.sceneLoaded` in `OnEnable` e si disiscrive in `OnDisable`. Ad ogni caricamento di scena, `ResetEvents()` azzera tutti i delegati, garantendo che il nuovo GameManager parta sempre da uno stato pulito.

Cosa non è cambiato Gli eventi pubblici e i metodi `Publish*` sono rimasti identici. Il contratto pubblico della classe non è stato toccato — chi già usava GameEvents non deve modificare nulla.

File `GameManager.cs`

Il Problema: La "God Class" Il vecchio `GameManager` violava il principio SRP accentrando tre responsabilità: logica di gioco, gestione delle scene e manipolazione della UI visiva. Questo forte accoppiamento rendeva il codice fragile: una modifica estetica a un bottone rischiava di intaccare le condizioni di vittoria.

La Soluzione: Divide et Impera Abbiamo scisso il monolite creando due componenti altamente specializzati:

- **`GameManager` (Il Cervello):** Mantiene il controllo esclusivo sulle regole (timer, monete, vittoria/sconfitta) e sulla navigazione. È diventato "cieco": ignora totalmente l'esistenza di Canvas, testi o immagini.
- **`UIManager` (Il Volto):** Gestisce unicamente la presentazione. Si fa carico della formattazione visiva (conversione MM:SS, timer rosso sotto i 15 secondi) e dell'interfaccia. È "ignorante": non sa come si vince o come si caricano le scene.

Il Disaccoppiamento: Il "Ponte Radio" GameEvents I due script sono isolati e non hanno riferimenti diretti l'uno dell'altro. La comunicazione avviene in modo bidirezionale ma anonimo tramite l'hub centrale:

- **Flusso Dati (Game -> UI):** Il `GameManager` calcola e pubblica eventi (es. `OnGameOver`). L'`UIManager` li intercetta e aggiorna lo schermo (mostra `_defeatImage`).
- **Flusso Comandi (UI -> Game):** Sono stati introdotti nuovi eventi (`OnBackToMenuRequested`, `OnRestartRequested`, inseriti nel `ResetEvents`). L'`UIManager` rileva il click fisico e pubblica l'evento; il `GameManager` lo ascolta e agisce su `SceneManager`.

Cosa NON è cambiato: Il nucleo matematico e i metodi di logica pura del `GameManager` (es. `CalculateNewCoinCount`) sono rimasti intatti, garantendo che il sistema continui a essere testabile in totale isolamento dall'Engine.

Bug Fix — GameManager/UIManager/GameEvents

Problema 1: ResetEvents() distruggeva le iscrizioni agli eventi al cambio scena

- **Causa:** La classe `GameEvents` (impostata come DontDestroyOnLoad) chiamava il metodo `ResetEvents()` durante `OnSceneLoaded`. Questo azzerava tutti gli eventi portandoli a null prima che i nuovi `GameManager` e `UIManager` della scena caricata avessero il tempo di iscriversi tramite il loro metodo `Start()`.
- **Soluzione:** Sono stati rimossi i metodi `ResetEvents()`, `OnSceneLoaded()`, `OnEnable()` e `OnDisable()` da `GameEvents.cs`. La responsabilità della disiscrizione è stata delegata esclusivamente agli script che si iscrivono, utilizzando il metodo `OnDestroy()`.

Problema 2: UIManager causava il doppio innesco degli eventi (Double Trigger)

- **Causa:** Lo script `UIManager` si iscriveva agli eventi `OnBackToMenuRequested` e `OnRestartRequested` all'interno del proprio metodo `SubscribeToEvents()`. Tuttavia, essendo lo stesso `UIManager` a pubblicare (invocare) questi eventi al click dei relativi bottoni, si generava una catena ricorsiva di chiamate indesiderate.
- **Soluzione:** Le iscrizioni a `OnBackToMenuRequested` e `OnRestartRequested` sono state rimosse dai metodi `SubscribeToEvents()` e `UnsubscribeFromEvents()` in `UIManager.cs`.

Problema 3: Incoerenza nell'attivazione e disattivazione dei pannelli UI

- **Causa:** Nel metodo `Start()` di `UIManager`, veniva utilizzato `_victoryImage.enabled = false`. Questo disabilitava unicamente il componente Image (rendendolo invisibile ma mantenendo il GameObject attivo), mentre il metodo `HandleGameOver` utilizzava successivamente `SetActive(true)` sull'intero GameObject, creando un comportamento asimmetrico.
- **Soluzione:** L'istruzione `_victoryImage.enabled = false` è stata sostituita con `_victoryImage.gameObject.SetActive(false)`, allineando la logica di inizializzazione con quella di attivazione presente in `HandleGameOver`.

Problema 4: Riferimenti errati nell'Inspector per i bottoni di fine partita

- **Causa:** Il campo `Back To Menu Button Victory` esposto nell'Inspector di `UIManager` era stato erroneamente assegnato al bottone "Back" (figlio del pannello `You_Lose_Background`), anziché al bottone corrispondente nel pannello `You_Win_Background`.
- **Soluzione:** Il riferimento nell'Inspector è stato corretto assegnando il bottone "Win" figlio del GameObject `You_Win_Background`.

Lezione Architetturale Il pattern corretto quando si utilizza un Singleton marcato con `DontDestroyOnLoad` in combinazione con un sistema ad eventi è il seguente: chi si iscrive deve gestire in autonomia la propria disiscrizione. Il publisher (la classe che detiene e lancia gli eventi) non deve mai azzerare i propri eventi manualmente. Ogni subscriber (chi ascolta l'evento) deve utilizzare il proprio metodo `OnDestroy()` per rimuovere l'iscrizione, garantendo così una pulizia automatica al cambio scena.

File `PlayerController.cs`

Questo refactoring ottimizza le performance e allinea il codice al corretto ciclo di vita di Unity. E' stato **cachato `Camera.main` in `Awake`** per impedire all'engine di eseguire una costosa ricerca nella scena a ogni singolo frame. Sempre in `Awake`, E' stato **spostato i `GetComponent`** (come `_rb` e `_health`), assicurando che i componenti interni del giocatore siano inizializzati e pronti *prima* che altri script esterni tentino di interagirvi durante i loro `Start`. Infine, E' stato rimosso il "rumore" dal codice: **eliminando l'inutile controllo `if`** prima di `freezeRotation` (poiché assegnare un valore a `true` quando lo è già non ha alcun costo computazionale rispetto al valutare una condizione) e **rimuovendo gli `[Header]`** posti su variabili private non serializzate, che risultavano invisibili nell'Inspector e quindi puramente fuorvianti.

File `PlayerHealth.cs`

Il codice è stato ottimizzato seguendo i principi di **Clean Code**, le **Naming Convention** standard di C# e la gestione corretta del **Lifecycle di Unity**.

1. Inizializzazione e Lifecycle (Awake vs Start)

Abbiamo separato l'impostazione dello stato interno dalla comunicazione esterna.

- **In `Awake()`:** Inizializziamo lo stato interno (`_currentHealth = _maxHealth`). Questo garantisce che l'oggetto sia "pronto" prima che chiunque altro possa interagirvi.
- **In `Start()`:** Invochiamo l'evento iniziale `OnHealthChanged`. In questo momento siamo certi che tutti gli altri script (es. la UI) abbiano completato il loro `Awake` e si siano registrati come ascoltatori (subscriber).

Principio: Awake = Preparazione interna; Start = Comunicazione esterna.

2. Coerenza del Naming (`_controller`)

È stata corretta la variabile da `controller` a `_controller`.

- **Perché:** L'uso del prefisso underscore (_) per i campi privati è la convenzione standard in C#. Permette di distinguere istantaneamente una variabile di classe da una variabile locale o da un parametro di un metodo, migliorando drasticamente la velocità di lettura del codice.

3. Ottimizzazione Logica (Mathf.Clamp e Ternario)

Abbiamo pulito la sintassi per renderla più fluida e meno soggetta a stati intermedi invalidi.

- **Clamp Inline:** `_currentHealth = Mathf.Clamp(_currentHealth - damage, 0, _maxHealth);` — Invece di modificare la variabile e poi correggerla, calcoliamo il valore finale in un'unica operazione atomica.
- **Operatore Ternario:** In `SetMaxHealth`, abbiamo usato `condizione ? vero : falso`. È ideale per assegnazioni semplici, rendendo il codice più compatto senza perdere chiarezza rispetto a un blocco `if-else` esteso.

4. Pulizia del Codice (Rimozione Debug.Log)

I log di tracciamento nelle funzioni `TakeDamage` e `Heal` sono stati rimossi.

- **Motivazione:** In produzione, le stringhe e i log generano "spazzatura" in memoria (Garbage Collection) e rallentano il processore. Un sistema di salute professionale deve essere silenzioso e comunicare il suo stato solo tramite eventi (`Action`).

Bug `GameEvents is null`

`PlayerHealth.Awake()` cachava `GameEvents.Instance` troppo presto — l'ordine di `Awake` tra GameObject non è garantito in Unity, quindi `Instance` era ancora null. Fix: recuperare `GameEvents.Instance` direttamente dentro `DelayedGameOver()` al momento dell'uso, quando tutti gli `Awake` sono già stati eseguiti.

File `HealthBar.cs`

Il sistema di visualizzazione della salute è stato reso più robusto e reattivo, garantendo la sincronizzazione dei dati e la fluidità visiva.

1. Lifecycle e Sottoscrizione Eventi (`OnEnable` / `OnDisable`)

Abbiamo spostato la logica di iscrizione agli eventi da `Start` a `OnEnable`.

- **Perché:** Permette alla barra di funzionare correttamente anche se il GameObject viene disattivato e riattivato (es. menu di pausa o pooling). La disiscrizione in `OnDisable` previene i "Memory Leak" e gli errori di riferimento nullo quando l'oggetto viene distrutto o disattivato.
- **Correzione:** Ripristinata la sintassi corretta `+=` per i delegati C#, sostituendo i commenti errati.

2. Sincronizzazione Immediata allo Stato Iniziale

All'interno di `OnEnable`, dopo la sottoscrizione, viene chiamata immediatamente `UpdateHealthBar`.

- **Risoluzione:** Questo risolve il classico problema temporale in Unity: se il Player inizializza la salute in `Start` prima che la barra sia pronta, la UI rimarrebbe vuota o piena per errore. Chiamando la funzione manualmente all'abilitazione, la barra si allinea istantaneamente ai dati attuali del Player.

3. Logica di Animazione e Interpolazione

È stato identificato e corretto il problema della mancata animazione.

- **Correzione:** Introdotto l'uso di `Mathf.MoveTowards` (o Lerp) all'interno del metodo `Update`. Senza questa logica, il valore `_targetFillAmount` calcolato dall'evento rimaneva un dato isolato senza mai influenzare l'effettivo `fillAmount` dell'immagine.
- **Risultato:** Transizione fluida della barra che migliora il feedback visivo per il giocatore.

4. Incapsulamento e Robustezza

- **Visibilità dei Metodi:** `UpdateHealthBar` è stato reso **private**. Seguendo il principio di incapsulamento, questo metodo deve essere attivato solo dall'evento interno della classe e non deve essere manipolabile da script esterni, evitando interferenze logiche.
- **Error Reporting:** Sostituito `Debug.Log` con `Debug.LogError` per i riferimenti mancanti (`_playerHealth`). In un contesto professionale, un componente UI senza i suoi dati è un errore critico che deve bloccare l'attenzione dello sviluppatore immediatamente.

File `CameraOrbit.cs`

- Implementare CineMachine

Feedback Scuola

- è inutile fare l'if in cui controlli se il freezeRotation non è true, per poi settarlo a true, settalo direttamente a true... tanto l'intento è comunque di forzarlo a true ✅
- Il caricamento delle scene via indice non è il massimo, perchè se cambia l'ordine in build settings verrebbe caricata la scena sbagliata✅
- Nel MainMenu è presente una canvas di troppo ✅
- In TurretBullet per distruggere i proiettili utilizzi un sistema un po' troppo strano e che alla lunga potrebbe farti concatenare un sacco di if() else if() ✅
- Sempre in TurretBullet prendi il componente PlayerHealth 2 volte ✅
- C'è una quantità di commenti veramente troppo esagerata che rende il codice poco leggibile, inoltre tutti i commenti sono stati scritti chiaramente da una IA ✅
- Inoltre sembra che questo progetto, da come sono scritti i commenti e tutto, sia un' API e non un gioco ✅
- Alcuni passaggi non mi tornano sembra che prima fosse stato fatto tutto usando i singleton e successivamente l'IA avesse fatto un cambio da Singleton a "Event Driven System" tra l'altro non implementato correttamente, implementare una cosa del genere porta con se dei side effects che bisogna conoscere ✅
- In Coin utilizzando quel tipo di sistema ad eventi fai una ricerca con FindObjectOfType ogni volta che viene preso un coin ✅
- In PlayerController prendi il camera forward e right chiamando ad ogni frame 2 volte Camera.main che internamente fa una ricerca abbastanza costosa su tutti i gameobject alla ricerca del tag MainCamera, conviene sempre salvarsi una variabile cameraMain e settarla in Awake() ✅
- In HealthBar se attivi animateChanges aggiorneresti la UI in update continuamente e questo è costosissimo

Correzioni applicate Feedback Scuola

- è inutile fare l'if in cui controlli se il freezeRotation non è true, per poi settarlo a true, settalo direttamente a true... tanto l'intento è comunque di forzarlo a true
  Ho corretto questo feedback ed è visibile nella riga 62 di [PlayerController.cs]

```
        _rb.freezeRotation = true;
```

- Il caricamento delle scene via indice non è il massimo, perchè se cambia l'ordine in build settings verrebbe caricata la scena sbagliata
  Ho corretto questo feedback ed è visibile nella riga 185, 198 di [GameManager.cs]

```
// SceneManager.LoadScene(0);
        SceneManager.LoadScene("MainMenu");
//
SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex);
        SceneManager.LoadScene("Level1");
```

- Nel MainMenu è presente una canvas di troppo

  Canvas Eliminato
- In TurretBullet per distruggere i proiettili utilizzi un sistema un po' troppo strano e che alla lunga potrebbe farti concatenare un sacco di if() else if()

  Applicati Layers che mi hanno permessi di snellire il codice di TurreBullet
- C'è una quantità di commenti veramente troppo esagerata che rende il codice poco leggibile, inoltre tutti i commenti sono stati scritti chiaramente da una IA

  I Commenti eccessivi sono stati cancellati, i commenti mi servono per tenere traccia degli aggiornamenti fatti.
- Inoltre sembra che questo progetto, da come sono scritti i commenti e tutto, sia un' API e non un gioco

  E' una metodologia che applico io ai miei progetti, capisco la questione dei commenti generati da IA e aiuto di questa nello scrivere codice, ma non comprendo il feedback riguardo la metodologia di Debug o che questo progetti sembri API piuttosto che un videogioco. Sembra più una critica…
- Alcuni passaggi non mi tornano sembra che prima fosse stato fatto tutto usando i singleton e successivamente l'IA avesse fatto un cambio da Singleton a "Event Driven System" tra l'altro non implementato correttamente, implementare una cosa del genere porta con se dei side effects che bisogna conoscere

  Il progetto è diventato tolamente EventDriven, nei miei limiti è stato applicato. Il cambio da Singleton a Event System è stato fatto sotto richiesta del docente Zuliani
- In Coin utilizzando quel tipo di sistema ad eventi fai una ricerca con FindObjectOfType ogni volta che viene preso un coin

  Risolto cachando in Awake

```
private GameEvents _gameEvents;

    private void Awake()
```

```
    {
        // Cerchiamo il componente GameEvents nella scena
        _gameEvents = GameEvents.Instance;
    }
```

- In HealthBar se attivi animateChanges aggiorneresti la UI in update continuamente e questo è costosissimo

  Il ridisegno della UI in questo caso era un calcolo costoso, è stata applicata la Approximately, che aiuta nel controllare `fillAmount` *ha raggiunto* `_targetFillAmount`, *Unity non ridisegna la UI.*